

PDDLego: DomainBridging the Gap Between Classical and Abstract Reasoning in a LEGO-Based Benchmark

Zeynel, Marian, Steffen

July 23, 2026

**RWTH: Artificial Intelligence for Robotics
Proposal
Project 1: PDDL Modeling for WorkBenchMark**

1 Outline

This project is based on the **WorkBenchMark** benchmark and deals with LEGO assembly tasks. Originally, an Assembly-by-Disassembly (ABD) planner is used and consequently represents the baseline for our project.

We plan to design a Planning Domain Definition Language (PDDL) domain for a subset of the available tiers of LEGO Duplo assembly tasks coupled with a general-purpose PDDL planner, as proposed in the document for the final projects of this Robotics course.

Subsequently, we plan to analyze the resulting planning and execution capacities enabled by the PDDL domain, PDDL planner and the **DomainBridge** provided by **TAMPanda**.

2 Implementation

The following list contains the different parts of our implementation that are necessary to fulfill the above described goals.

PDDL Domain File. We will define the framework for a fitting PDDL domain, including types, predicates and actions.

Python Parser. We plan to provide a parser, implemented in the Python programming language, to map **WorkBenchMark** YAML specifications to PDDL problem files.

UPF. We plan to provide an integration with the Universal Planning Framework (UPF) for ordered assembly plans.

TAMPanda and DomainBridge. We will provide a mapping from PDDL actions to **PickPlaceExecutor** calls with **PAMPanda** and **DomainBridge** to allow for the execution of plans.

3 Limitations

To ensure that the scope of the project is adequately limited according to the resources and time available to us, we decide on the following limitations:

User Interface. We forgo the implementation of an user interface as the number of possible selected factors is limited (*i.e.*, chosen tier, chosen number of tasks) and allow the execution of our implementation purely through the terminal.

Selection of Tiers. We limit the number of selected tiers (tier 1 and tier 2) and thus restrict the difficulty of tasks that are subject to our implemented planning and execution procedure.

Simulation-Based. We do not plan to use a real robot to execute the constructed plans, and instead use the MuJoCo simulation environment.

Ground-Truth Object Positions. We directly access the ground-truth object positions and forgo any geometric sensing or similar methods.

4 Evaluation

We evaluate our proposed project based on several factors and evaluation metrics, as explained in the following:

Planning Success Rate. Let n be the number of overall tasks and p be the number of tasks for which our implementation successfully finds a plan, then we define the planning success rate as $\frac{p}{n}$.

Execution Success Rate. Let p be the number of tasks for which our implementation successfully finds a plan and e be the number of tasks for which the plan is successfully executed, then we define the execution success rate as $\frac{e}{p}$.

Planning Time. Captures the time from the beginning to the end of the planning procedure, expressed in seconds. For calculation, we will exclude any factors that do not directly contribute to the planning, such as the parsing from YAML specification to PDDL problem file to guarantee fair comparisons.

Plan Length. We define the plan length as the number of steps contained in the generated plan and use it as a measure of efficiency.

Tier Comparison. We compare the above defined metrics across the two different tiers, in order to estimate the behavior of the planner when faced with increasingly difficult tasks.

PDDL Domain Analysis. We analyze the expressivity of our PDDL Model with respect to its displayed performance (*i.e.*, overspecification, underspecification, symbolic granularity, choice of facts).

Planner Comparison. We compare all of the above mentioned metrics to the ABD baseline, if applicable. This way, we gain a clear understanding of the strengths but also limitations of the PDDL planner and our implementation approach.

Failure Analysis. Finally, we analyze the failures and pitfalls of our approach with respect to different aspects such as the structure of tasks, difficulty of tasks, choice of predicates and various other factors. Then, we conclude with possible remedies in future iterations of our implementation.

Using this collection of metrics and evaluation approaches we hope to gain a deeper, empirical understanding of the behavior of our implementation and to contribute to the science of planning and automation in a meaningful way.